Candidate Transformer AI

Design Document · Spring Boot 3 · Java 21 · Apache PDFBox · Thymeleaf

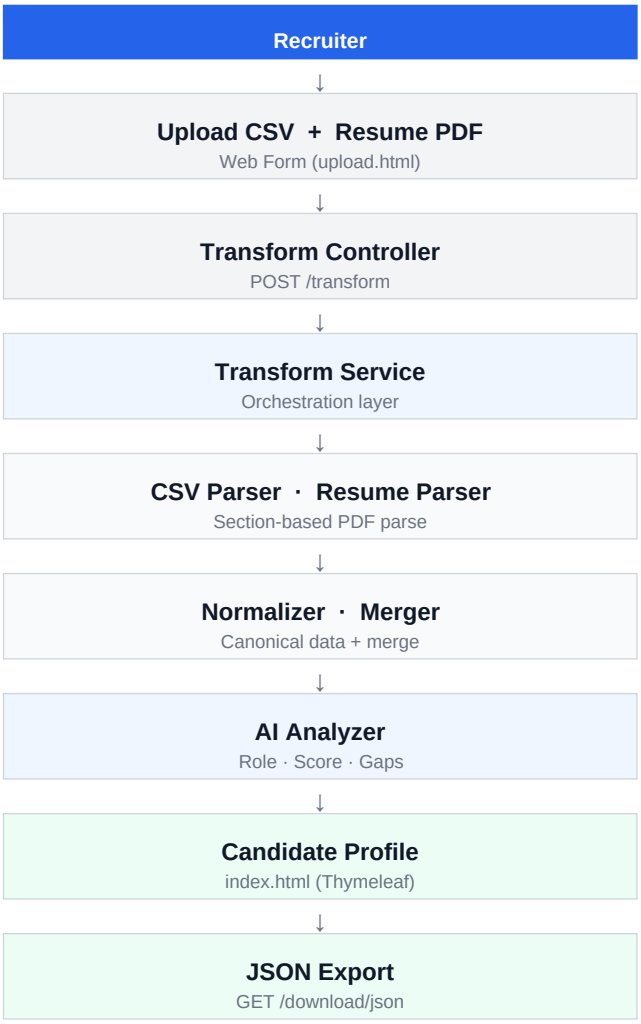
■ 1. Project Overview

Candidate Transformer AI is an ATS-style recruiter tool that eliminates manual resume screening. Recruiters upload a structured CSV containing candidate metadata and an unstructured Resume PDF. The application automatically parses, normalises, merges, and AI-analyses both sources — generating a unified candidate profile and a recruiter-friendly JSON report suitable for sharing with hiring managers.

■ 2. Technology Stack

Layer	Technology
Backend	Java 21, Spring Boot 3, Spring MVC
Frontend	Thymeleaf, Bootstrap 5, HTML/CSS
PDF Parsing	Apache PDFBox 2.0.30
CSV Parsing	Apache Commons CSV 1.10.0
JSON	Jackson Databind
Build Tool	Maven (Spring Boot Maven Plugin)
Server	Embedded Apache Tomcat

■ 3. System Architecture



■ 4. End-to-End Workflow

1. Upload — Recruiter submits CSV + Resume PDF via web form

2. Parse — CsvParser maps CSV headers; ResumeParser segments PDF sections

3. Normalise — Names title-cased; emails lowercased; phones standardised (+91)

4. Merge — CSV metadata and resume content unified into one Candidate object

5. Analyse — AIAnalyzer scores skills across 9 role clusters; computes match score

6. Profile — Thymeleaf renders recruiter-friendly candidate report in browser

7. Export — DownloadController serves JSON via CandidateReportDto (DTO pattern)

■ 5. Design Decisions

Section-based PDF parsing
Segments the resume into named sections before extraction, preventing content from one section leaking into another.

Strict whitelist skill extraction
Skills are matched only against a curated ~120-entry technology list, eliminating certificate names and project descriptions from skill output.

Separation of parsing, normalisation, and merging
Each responsibility is a dedicated class (CsvParser, ResumeParser, Normalizer, Merger), making the pipeline independently testable.

Rule-based AI analyser (no external API)
Role detection, match scoring, and gap analysis run entirely in-process, ensuring zero latency, offline operation, and no API cost.

DTO pattern for JSON export
The internal Candidate model is never serialised directly; RecruiterJsonBuilder assembles a clean CandidateReportDto, keeping recruiter output decoupled from implementation details.

Resume always preferred over CSV for structured data
Education, experience, projects, and certifications are always taken from the resume; CSV contributes metadata (headline, location) only.

■ 6. Edge Cases Handled

- Fresher resume with no work experience → yearsExperience = 0, labelled "Fresher"
- Resume with no Skills section → empty skill list returned, no guessing
- Certificate names adjacent to Skills section → stop condition prevents extraction
- Education dates → never used for experience calculation (dedicated section check)
- Missing email or phone → empty list, not null; no crash in view layer
- Duplicate skills → deduplicated alphabetically before display
- Missing CSV field → graceful fallback returns empty string via getValue()
- No candidate in session → HTTP 400 returned with readable error message

■ 7. Future Improvements

- Job Description Matching — score each candidate against a specific JD for ranked fit.
- LLM-powered summary — replace rule-based summary with GPT-4 generated narrative.
- Batch resume processing — allow multiple PDF uploads in a single session.